

Name:	Abdul Rehman
Track	Advanced AI & Automation
Weeks	6 & 7

1. Overview

A threat intelligence platform helps a Security Operations Center turn large amounts of security data into information that analysts can actually use. Instead of treating every IP address, domain, URL, or file hash as a separate alert, the platform collects indicators, checks them against several sources, assigns a risk level, and sends useful results to the systems responsible for detection and response.

For this design, the platform brings together internal security data, external threat intelligence feeds, an enrichment and scoring layer, IOC storage, MISP, and the SOC tools used for response. The design also takes into account three practical problems: API limits, old or expired indicators, and false positives.

2. Threat Data Sources

The first part of the platform is the data ingestion layer. It collects information from both inside and outside the organization.

Internal sources provide information about what is actually happening in the environment. These include SIEM alerts and logs, EDR/XDR activity, firewall and proxy logs, Zeek network data, and honeypots. For example, a honeypot can identify IP addresses that are scanning the organization's network.

External sources provide additional context about indicators seen elsewhere. These can include OSINT feeds such as Spamhaus and abuse.ch, CERT advisories, sector information-sharing groups, and public threat intelligence APIs such as VirusTotal, AbuseIPDB, and AlienVault OTX.

Using both types of sources is important. External feeds provide broader context, while internal telemetry helps determine whether an indicator is relevant to the organization's own environment.

3. Normalization, Enrichment and Caching

Before indicators are stored or passed to other systems, they need to be cleaned and normalized. The platform converts incoming data into common types such as IP addresses, domains, URLs, and file hashes. Duplicate indicators are removed so the same IOC does not repeatedly enter the system.

The enrichment engine then checks indicators against the available threat intelligence sources. A two-level cache is used to reduce unnecessary API requests. A local in-memory cache handles frequently requested information quickly, while a shared Redis cache stores API responses for longer periods. In this design, the shared cache keeps responses for 12 hours.

When an indicator is not already cached, the system queries VirusTotal, AbuseIPDB, and OTX. The results are combined into a threat score from 0 to 100. VirusTotal has a weight of 0.40, AbuseIPDB 0.35, and OTX 0.25. If the indicator is also detected by internal EDR or honeypot telemetry, an additional internal boost can be applied, with the final score capped at 100.

4. IOC Storage and Standards

The platform uses different types of storage because the information being collected has different needs. PostgreSQL can store structured IOC information such as confidence, timestamps, access control information, and audit records. MongoDB can hold larger or less structured data, including raw API responses and investigation details. Redis is used for fast lookups, caching, rate limiting, and queues.

The platform also follows STIX 2.1 and TAXII 2.1 so that threat intelligence can be represented and exchanged in a standard format. MISP is used as the main threat intelligence sharing and correlation component. This makes it easier to organize indicators, apply warninglists and taxonomies, and share relevant intelligence with trusted organizations.

5. Handling Rate Limits and API Failures

External services have different request limits, so the platform should not send requests without control. Redis can maintain request counters and apply a token-bucket rate limiter for each API. Requests can also be placed into priority queues so an indicator involved in an active incident is handled before routine background enrichment.

If an API responds with a rate-limit error such as HTTP 429, the system should wait before retrying. Exponential backoff with a small amount of random delay helps prevent repeated requests from immediately hitting the same limit. Caching is the first line of defense because it avoids making the same API request again.

The enrichment engine should also handle timeouts, unavailable APIs, and invalid responses without stopping the rest of the pipeline. If one source fails, the other available sources can still be used and the failure can be recorded for later review.

6. IOC Expiration and False-Positive Prevention

Threat intelligence becomes less useful when indicators are kept active forever. The platform therefore uses confidence decay and expiration rules. Scanner and command-and-control IP addresses have a short half-life of 3 days and a maximum lifetime of 14 days. Phishing domains and URLs have a 7-day half-life and a maximum lifetime of 30 days. Malware SHA-256 hashes have a much longer half-life of 180 days and can be retained permanently. Indicators whose confidence falls below 30 can be removed from active firewall lists and archived.

False positives are handled before automatic blocking. Common trusted domains, CDNs, cloud provider ranges, and private RFC 1918 addresses can be checked against warning or allow lists. Automatic blocking should normally require confirmation from at least two independent external sources or supporting internal telemetry. Analyst feedback should also be used to reduce the confidence of indicators that were incorrectly flagged.

7. SOC Workflow and MISP Integration

Once an indicator has been enriched and scored, the result can be sent to the systems that need it. High-confidence indicators, such as those with a score of 80 or above, can be pushed to firewall and EDR blocklists. The indicators can also be synchronized with the SIEM for correlation with new logs and alerts.

SOAR tools can use the intelligence to automate response actions, such as isolating a host or opening a ticket for an analyst. MISP provides the sharing and correlation layer, including taxonomies, warninglists, relationship information, and sharing controls such as Traffic Light Protocol (TLP) tags.

The overall flow is straightforward: security data enters the platform, indicators are normalized and enriched, results are scored and stored, and useful intelligence is then sent to the SOC for detection, blocking, investigation, and response.

8. Architecture Diagram

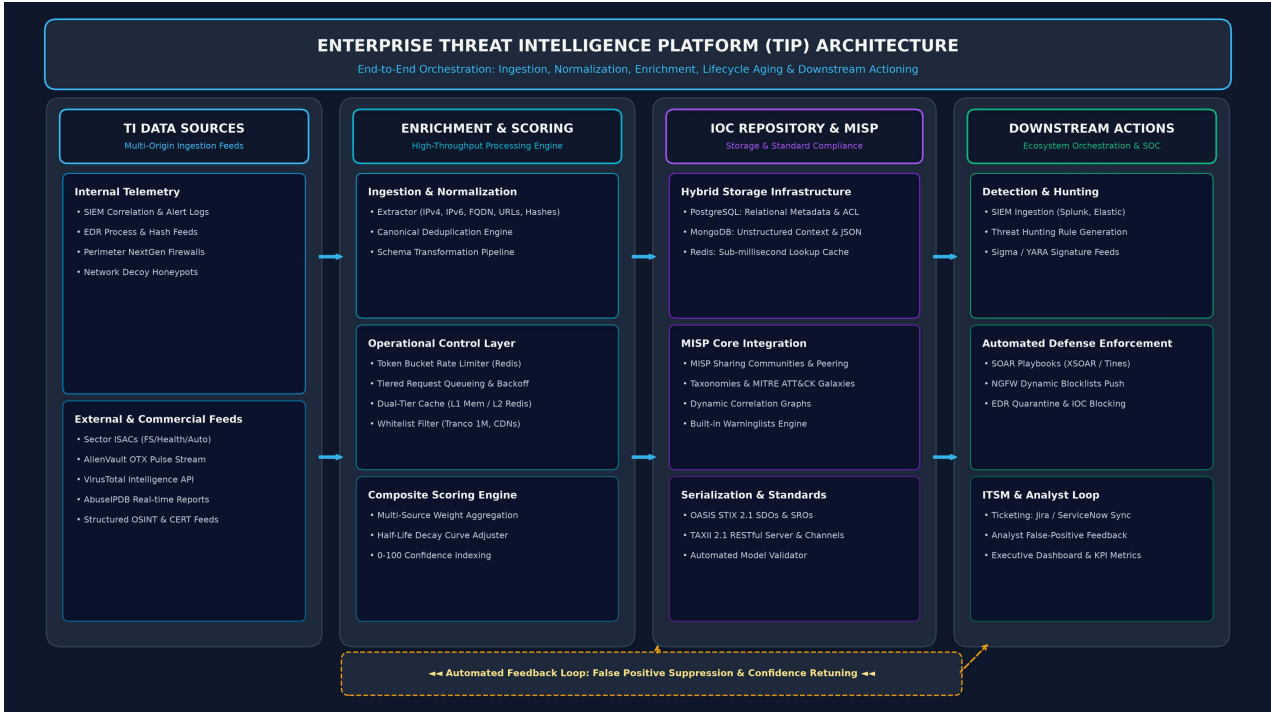


Figure 1. End-to-end threat intelligence platform flow

9. Main Components

Area	Technology	Purpose
Ingestion	Kafka / Celery	Collect and process security data
Caching and rate limiting	Redis	Cache API responses and control requests
Storage	PostgreSQL / MongoDB	Store structured and raw IOC information
Threat sharing	MISP / STIX / TAXII	Organize and exchange threat intelligence
SOC integration	SIEM / SOAR / NGFW	Use intelligence for detection and response